

Kazakh-British Technical University

Data collection and preparation

Laboratory work №1 Reading and Converting Formats

Full Name: Baibolov

Sultanmakhmut

ID: 23B031235

08.09.2026

Almaty

Table of contents

Task 1. Dataset selection	3
Task 2. Data-quality analysis	4
Task 3. Data formats	5
Task 4. Data format comparison	6

Task 1. Dataset selection

Select an open dataset from Kaggle, Data.gov, World Bank Open Data, UN Data or another open source. The dataset must contain at least 100 rows, five columns and at least one data-quality problem.

1. Dataset name and source link

Dataset name - Netflix Movies and Series

Source link - <https://www.kaggle.com/datasets/debayank2024/netflix-movies-and-series?resource=download>

2. Author

Author - Debayan Bandyopadhyay (<https://www.kaggle.com/debayank2024>)

3. Number of rows and columns

Rows – 8807

```
[22]: len(titles_table.index)
```

```
[22]: 8807
```

```
[26]: titles_table.index
```

```
[26]: RangeIndex(start=0, stop=8807, step=1)
```

Columns - 12

```
[25]: len(titles_table.columns)
```

```
[25]: 12
```

```
[24]: titles_table.columns
```

```
[24]: Index(['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added',  
        'release_year', 'rating', 'duration', 'listed_in', 'description'],  
        dtype='str')
```

4. Meaning of 1 row and each column

```
[27]: titles_table.iloc[0]
```

```
[27]: show_id          s1  
      type          Movie  
      title          Dick Johnson Is Dead  
      director      Kirsten Johnson  
      cast          NaN  
      country      United States  
      date_added    September 25, 2021  
      release_year    2020  
      rating        PG-13  
      duration      90 min  
      listed_in      Documentaries  
      description    As her father nears the end of his life, filmm...  
      Name: 0, dtype: object
```

- **show_id** –title id,

- **type** –title type (Movie/TV Show),
- **title** –name of the title,
- **director** –directors separated by commas (NaN if unknown),
- **cast** – list of actors separated by commas,
- **country** – production country,
- **date_added** – the Netflix adding date,
- **release_year** – release year,
- **rating** – content rating (https://en.wikipedia.org/wiki/Content_rating),
- **duration** – duration of the title (“X min” for movies and “X Seasons” for TV Shows),
- **listed_in** – title genres and tags (e. g. Dramas, International Movies),
- **description** – title description.

5. Dataset type

CSV file is designed to store data in tabular format, so dataset type is structured.

Task 2. Data-quality analysis

Identify the data types, missing values, duplicates, invalid values, inconsistent categories, incorrect date or number formats, and possible outliers. Present the results in a table containing the column name, identified problem, example, and its possible effect on further data analysis.

```
[92]: data_quality_analysis = pd.DataFrame(
        columns=titles_table.columns,
        index=["data type", "missing_values", "duplicates", "invalid values", ]
    )
    data_quality_analysis
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
data type	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
missing_values	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
duplicates	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
invalid values	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
[95]: data_quality_analysis.loc["data type"] = titles_table.dtypes

data_quality_analysis.loc["missing_values"] = titles_table.isna().astype(int).sum()

data_quality_analysis.loc["duplicates"] = titles_table.apply(lambda col: col.duplicated(), axis="index").astype(int).sum()

duration_pattern = r"\d+\s+(?:min|Seasons?)"
invalid_duration_mask = titles_table["rating"].str.match(duration_pattern)
data_quality_analysis.at["invalid values", "rating"] = len(titles_table[invalid_duration_mask])

# invalid values were calculated only for rating column as it was already known it has data entry error (values from duration column)
data_quality_analysis
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
data type	str	str	str	str	str	str	str	int64	str	str	str	str
missing_values	0	0	0	2634	825	831	10	0	4	3	0	0
duplicates	0	8805	0	4278	1114	8058	7039	8733	8789	8586	8293	32
invalid values	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	3	NaN	NaN	NaN

Feather format reading and writing is much faster than csv, also occupying less storage.

Column name	Identified problem	Example	Possible effect on further data analysis
date_added	Data type is str, string representation is unusual	"September 25, 2021"	Incorrect sorting, validation errors, broken date manipulation
rating	Data entry error	"74 min"	Validation error, data inconsistency
director, cast, country, date_added, rating, duration	Missing values	s5541: duration - NaN	Bias, reduced statistical power

Data problems analysis table

Task 3. Data formats

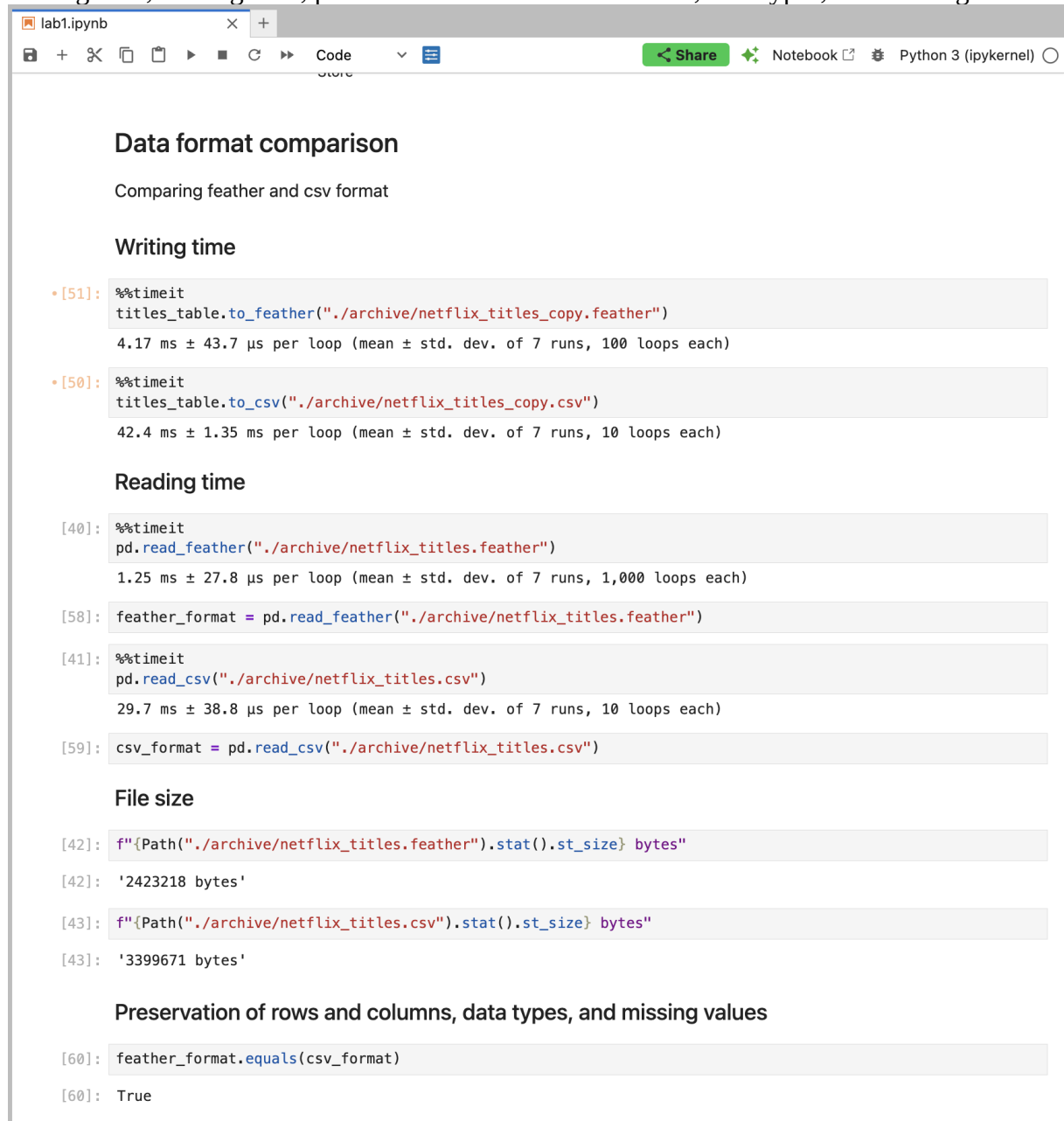
Save the prepared dataset in one suitable format and one alternative format. Compare CSV, JSON, Parquet, ORC, and Avro according to their representation, storage structure, schema support, human readability, typical applications, advantages, and limitations. Present the comparison to a table. Explain which format is the most appropriate for the selected dataset and justify your choice.

I chose the feather (Arrow IPC) format for fast file writing and reading to clearly demonstrate the difference between it and the csv format. The original dataset was in csv format, which I used to read the data. Also, the dataset is small and requires only one file reading operation, while all data analysis is done by the code, which makes it most suitable in my opinion.

File format / Criteria	CSV	JSON	Parquet	ORC	Avro
Representation	Comma separated values	Hierarchical list of JS objects	Columnar chunks with footer metadata	Columnar stripes with internal indexes	Binary row-based JSON records
Storage structure	Row-based	Hierarchical	Columnar	Columnar	Row-based
Schema support	No	No	Yes	Yes	Yes
Human readability	Yes	Yes	Yes	No	No
Typical applications	Data sharing between teams and loading to Excel table	Data exchange between apps via HTTP	Storing large amount of compressed data	Enterprise data processing	Real-time event streaming
Advantages	Universal, low write overhead	Human readable, flexible	Compression, fast queries	Compression, indexing	Compact, ultra-fast writes
Limitations	No schema, poor performance	High redundancy, slow parsing	Slow write, not human readable	Edit restrictions, slow write	Poor read, schema dependency

Task 4. Data format comparison

For the two formats used in your practical work, measure and compare file size, writing time, reading time, preservation of rows and columns, data types, and missing values.



The screenshot shows a Jupyter Notebook titled 'lab1.ipynb' with a toolbar at the top. The notebook content is as follows:

Data format comparison

Comparing feather and csv format

Writing time

```
[51]: %%timeit
titles_table.to_feather("./archive/netflix_titles_copy.feather")
4.17 ms ± 43.7 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)

[50]: %%timeit
titles_table.to_csv("./archive/netflix_titles_copy.csv")
42.4 ms ± 1.35 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

Reading time

```
[40]: %%timeit
pd.read_feather("./archive/netflix_titles.feather")
1.25 ms ± 27.8 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)

[58]: feather_format = pd.read_feather("./archive/netflix_titles.feather")

[41]: %%timeit
pd.read_csv("./archive/netflix_titles.csv")
29.7 ms ± 38.8 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)

[59]: csv_format = pd.read_csv("./archive/netflix_titles.csv")
```

File size

```
[42]: f"{Path('./archive/netflix_titles.feather').stat().st_size} bytes"
[42]: '2423218 bytes'

[43]: f"{Path('./archive/netflix_titles.csv').stat().st_size} bytes"
[43]: '3399671 bytes'
```

Preservation of rows and columns, data types, and missing values

```
[60]: feather_format.equals(csv_format)
[60]: True
```

Reading from the Feather (Arrow) file is 24 times faster, comparing to the CSV file format. Writing is 10 times faster. File size is smaller because it stores data in a binary format. Though performance of Feather in terms of speed and storage size is better than CSV, the obtained DataFrame objects are identical, meaning all rows and columns, data types and values are preserved.